

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №7
по дисциплине
«Программирование»

Выполнил студент гр. ИВТб-1303-06-00

_____/Гортоломей И.К./

Проверил преподаватель кафедры ЭВМ

_____/Баташев П.А./

Киров

2026

1 Цель и задачи работы

Цель работы: освоение практических навыков создания интерактивных дашбордов в Python, включая интеграцию `pandas DataFrame` с графическими библиотеками `matplotlib` и `seaborn`, встраивание графиков в окно `Tkinter` через бэкенд `FigureCanvasTkAgg`, реализацию событийно-ориентированной архитектуры (callbacks, event loop), динамическую фильтрацию, агрегацию и feature engineering «на лету», а также проектирование устойчивого процедурного кода без использования классов.

Задачи:

1. Сформировать и подготовить набор данных с категориальными и временными признаками.
2. Реализовать предобработку данных: очистку выбросов, клаппинг физических величин, расчёт скользящих средних и производных признаков.
3. Интегрировать графический движок Matplotlib/Seaborn с GUI Tkinter через адаптер `FigureCanvasTkAgg`.
4. Разработать панель управления с интерактивными элементами (выпадающие списки, переключатели агрегации, кнопки смены типа графика).
5. Реализовать механизм безопасного обновления данных через `.copy()` и перерисовки через `canvas.draw_idle()` без блокировки основного потока.
6. Внедрить экспорт визуализаций в PNG/PDF.

2 Описание варианта

Вариант №20: Водоподготовка

Предметная область: Мониторинг датчиков очистных сооружений. Контроль гарантирует соответствие санитарным нормам качества питьевой воды.

Основные метрики (поля датасета):

- `ts` (int32) – время замера (Unix timestamp);
- `filter_id` (int16) – идентификатор фильтра;
- `turb` (float32) – турбидность, NTU;
- `ph` (float32) – кислотность воды;
- `cl` (float32) – остаточный хлор, мг/л;
- `flow` (float32) – расход воды, м³/ч.

Условия фильтрации и очистки:

- `turb` $< 0 \rightarrow 0$; `cl` $< 0 \rightarrow 0$ (физическая невозможность отрицательных значений);
- `ph` обрезается (`clip`) по диапазону $[6.5; 8.5]$ согласно санитарным нормам;
- Выбросы по турбидности заменяются медианой группы (IQR-метод по `filter_id`);
- Редкие фильтры ($< 1\%$ от общего объёма) агрегируются в категорию 0.

Группировка и оконные функции:

- Группировка по `filter_id`: расчёт среднего `turb` (эффективность очистки) и максимума `flow` (нагрузка на систему);
- Скользящее окно $k = 35$: расчёт `turb_ma` для выявления долгосрочных трендов;

- `np.diff(flow)` → изменение расхода между замерами (индикатор работы насосов или засорения).

Акцент аналитики: Обратная зависимость чистоты воды и турбидности. При фильтрации аномалий каждый параметр обрабатывается независимо, без взаимного влияния масок.

Типы графиков: Линейный (динамика по неделям), столбчатый (сезонная агрегация), точечный (pH vs turbidity с цветовым кодированием статуса), гистограмма (распределение турбидности).

3 Листинг программного кода

Код реализован в процедурном стиле без использования классов. Логика разбита на этапы в соответствии с методическими указаниями.

Листинг 1: Этап 0-1: Инициализация и загрузка данных

```
1 import numpy as np
2 import pandas as pd
3 import tkinter as tk
4 from tkinter import ttk, messagebox, filedialog
5 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
    NavigationToolbar2Tk
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 from typing import Optional, List, Tuple
9 import warnings
10 warnings.filterwarnings('ignore')
11
12 # Глобальные переменные состояния
13 df_raw: Optional[pd.DataFrame] = None
14 df_processed: Optional[pd.DataFrame] = None # Обработанные данные
    (ОДИН РАЗ)
15 df_work: Optional[pd.DataFrame] = None      # Рабочая копия для
    графика
16 fig: Optional[plt.Figure] = None
```

```

17 canvas: Optional[FigureCanvasTkAgg] = None
18 current_chart: str = "line"
19 available_filters: List[int] = []
20
21 # Инициализация GUI и стилей
22 root = tk.Tk()
23 root.title("Дашборд: Интерактивный анализ")
24 root.geometry("1100x750")
25 root.configure(bg="#f0f2f5")
26
27 filter_var = tk.StringVar(value="Все")
28 agg_var = tk.StringVar(value="mean")
29
30 plt.rcParams['font.sans-serif'] = ['Arial', 'DejaVu Sans', '
    Liberation Sans']
31 plt.rcParams['axes.unicode_minus'] = False
32 sns.set_theme(style="whitegrid", palette="muted")
33
34 def load_data() -> Optional[pd.DataFrame]:
35     try:
36         df = pd.read_csv('data.csv', parse_dates=False)
37         if df.empty:
38             messagebox.showerror("Ошибка", "Файл data.csv пуст.")
39             return None
40         return df
41     except Exception as e:
42         messagebox.showerror("Ошибка", f"Не удалось загрузить CSV:
43             {e}")
44         return None
45
46 df_raw = load_data()
47 if df_raw is None:
48     raise SystemExit("Данные недоступны")

```

Листинг 2: Этап 2: Предобработка и Feature Engineering

```
1 def preprocess_data(df_input: pd.DataFrame) -> Tuple[pd.DataFrame,
2   List[int]]:
3     df = df_input.copy()
4     # 1. Замена NaN/Inf медианами и клаппинг физических границ
5     for col in ['turb', 'ph', 'cl', 'flow']:
6         clean_col = df[col].replace([np.inf, -np.inf], np.nan)
7         df[col] = clean_col.fillna(clean_col.median())
8     df['turb'] = df['turb'].clip(lower=0)
9     df['cl'] = df['cl'].clip(lower=0)
10    df['ph'] = df['ph'].clip(lower=6.5, upper=8.5)
11
12    # 2. Feature Engineering
13    df['dt'] = pd.to_datetime(df['ts'], unit='s')
14    df['ph_status'] = pd.cut(df['ph'], bins=[0, 6.5, 8.5, 14],
15                             labels=['ниже нормы', 'норма', 'выше
16                                     нормы']).astype('category')
17    df['season'] = pd.cut(df['dt'].dt.month, bins=[0, 3, 6, 9, 12],
18                          labels=['зима', 'весна', 'лето', 'осень',
19                                  ]).astype('category')
20    df['compliance_flag'] = (df['turb'] <= 1.5) & df['ph'].between
21                          (6.5, 8.5) & df['cl'].between(0.3, 0.5)
22
23    df = df.sort_values(['filter_id', 'ts']).reset_index(drop=True)
24
25    # Скользящее среднее (k=35) и тренд
26    df['turb_ma'] = df.groupby('filter_id')['turb'].transform(
27        lambda x: x.rolling(window=35, min_periods=1).mean()
28    )
29    df['turb_diff'] = df.groupby('filter_id')['turb_ma'].transform(
30        lambda x: x.diff().fillna(0))
31    df['turb_trend'] = np.sign(df['turb_diff']).map(
32        {-1: 'снижается', 0: 'стабильно', 1: 'растёт'})
33    ).astype('category')
34
35    # 3. Обрезка выбросов (IQR по группам)
```

```

31     grp = df.groupby('filter_id')['turb']
32     q1, q3 = grp.transform('quantile', 0.25), grp.transform('
    quantile', 0.75)
33     iqr = q3 - q1
34     lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
35     is_outlier = (df['turb'] < lower) | (df['turb'] > upper)
36     df.loc[is_outlier, 'turb'] = grp.transform('median')[is_outlier
    ]
37
38     # 4. Оптимизация: редкие фильтры -> 0
39     threshold = int(0.01 * len(df))
40     counts = df['filter_id'].value_counts()
41     rare_ids = counts[counts < threshold].index
42     non_rare_filters = sorted([f for f in df['filter_id'].unique()
    if f not in rare_ids and f != 0])
43     df.loc[df['filter_id'].isin(rare_ids), 'filter_id'] = 0
44     df['filter_id'] = df['filter_id'].astype('category')
45     return df, non_rare_filters
46
47 df_processed, available_filters = preprocess_data(df_raw)

```

Листинг 3: Этап 3: Встраивание Figure в Tkinter

```

1 plot_frame = tk.Frame(root, bg="white", relief=tk.SUNKEN, bd=1)
2 plot_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)
3
4 fig = plt.Figure(figsize=(10, 6), dpi=100)
5 canvas = FigureCanvasTkAgg(fig, master=plot_frame)
6 canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
7
8 toolbar = NavigationToolbar2Tk(canvas, plot_frame)
9 toolbar.update()
10 toolbar.pack(side=tk.TOP, fill=tk.X)

```

Листинг 4: Этап 4: Функции отрисовки графиков (Seaborn/Matplotlib)

```
1 def clear_figure() -> plt.Axes:
2     fig.clear()
3     return fig.add_subplot(111)
4
5 def plot_line() -> None:
6     ax = clear_figure()
7     if df_work is None or df_work.empty: return
8     df_plot = df_work.copy()
9     df_plot['week'] = df_plot['dt'].dt.to_period('W').dt.start_time
10    agg_df = df_plot.groupby(['week', 'filter_id'])['turb'].mean().
        reset_index()
11    for f_id in sorted(agg_df['filter_id'].unique()):
12        group = agg_df[agg_df['filter_id'] == f_id]
13        ax.plot(group['week'], group['turb'], label=f'Фильтр {int(
            f_id)}', linewidth=1.5, alpha=0.7)
14    ax.set_title("Динамика турбидности (агрегация по неделям)")
15    ax.legend(loc='upper left', bbox_to_anchor=(1.02, 1), ncol=2,
        fontsize=8)
16    fig.autofmt_xdate()
17    fig.tight_layout(rect=[0, 0.03, 0.88, 0.95])
18    canvas.draw_idle()
19
20 def plot_bar() -> None:
21     ax = clear_figure()
22     if df_work is None or df_work.empty: return
23     agg_df = df_work.groupby('season')['turb'].agg(agg_var.get()).
        reset_index()
24     agg_df.columns = ['season', f'turb_{agg_var.get()}']
25     sns.barplot(data=agg_df, x='season', y=f'turb_{agg_var.get()}',
26         palette=['#4A90E2', '#7ED321', '#F5A623', '#BD10E0'
27             ], ax=ax, errorbar=None)
28     ax.set_title(f"Турбидность по сезонам ({agg_var.get()})")
29     fig.tight_layout()
30     canvas.draw_idle()
```



```

31 def plot_scatter() -> None:
32     ax = clear_figure()
33     if df_work is None or df_work.empty: return
34     df_plot = df_work.sample(n=min(5000, len(df_work)),
35                                random_state=42)
36     sns.scatterplot(data=df_plot, x='ph', y='turb', hue='ph_status',
37                    , ax=ax, alpha=0.6, s=30,
38                    palette={'ниже нормы': 'blue', 'норма': 'orange',
39                             'выше нормы': 'red'})
39     ax.set_title("Зависимость турбидности от pH")
40     fig.tight_layout(rect=[0, 0.03, 0.88, 0.95])
41     canvas.draw_idle()
42
43 def plot_histogram() -> None:
44     ax = clear_figure()
45     if df_work is None or df_work.empty: return
46     sns.histplot(data=df_work, x='turb', kde=True, ax=ax, bins=50,
47                  color='steelblue', alpha=0.7)
48     ax.set_title("Распределение турбидности")
49     fig.tight_layout()
50     canvas.draw_idle()

```

Листинг 5: Этап 5-6: Панель управления и интерактивность

```

1 def apply_filters_and_redraw() -> None:
2     global df_work
3     if df_processed is None: return
4     df_work = df_processed.copy() # Безопасное копирование
5     selected = filter_var.get()
6     if selected != "Все":
7         df_work = df_work.loc[df_work['filter_id'] == int(selected)
8                                ].copy()
9
10    # Маршрутизация отрисовки
11    if current_chart == "line": plot_line()
12    elif current_chart == "bar": plot_bar()
13    elif current_chart == "scatter": plot_scatter()

```

```

13     elif current_chart == "histogram": plot_histogram()
14
15 def set_chart_type(chart_type: str):
16     global current_chart
17     current_chart = chart_type
18     apply_filters_and_redraw()
19
20 def export_plot() -> None:
21     filepath = filedialog.asksaveasfilename(defaultextension=".png"
22     ,
23                                           filetypes=[("PNG", "*.png"), ("PDF", "*.pdf")])
24
25     if filepath:
26         fig.savefig(filepath, dpi=300, bbox_inches='tight')
27         messagebox.showinfo("Экспорт", "График сохранён")
28
29 # Виджеты управления
30 ctrl_frame = tk.Frame(root, bg="#f0f2f5")
31 ctrl_frame.pack(fill=tk.X, padx=10, pady=8)
32
33 tk.Label(ctrl_frame, text="Фильтр:", bg="#f0f2f5").pack(side=tk.LEFT, padx=5)
34 filter_combo = ttk.Combobox(ctrl_frame, textvariable=filter_var,
35                             values=["Bce", 0] + [str(f) for f in
36                             available_filters],
37                             width=10, state="readonly")
38 filter_combo.pack(side=tk.LEFT, padx=5)
39 filter_var.trace_add("write", lambda *_: apply_filters_and_redraw())
40
41 for agg in ["mean", "median", "sum"]:
42     tk.Radiobutton(ctrl_frame, text=agg.capitalize(), variable=
43                     agg_var,
44                     value=agg, bg="#f0f2f5", command=
45                     apply_filters_and_redraw).pack(side=tk.LEFT,
46                     padx=2)

```

```

41
42 for chart, txt in [("line", "Линейный"), ("bar", "Столбчатый"),
43                  ("scatter", "Точечный"), ("histogram", "
44                      Гистограмма")]:
45     tk.Button(ctrl_frame, text=txt, command=lambda t=chart:
46               set_chart_type(t), width=10).pack(side=tk.LEFT, padx=4)
47
48 tk.Button(ctrl_frame, text="?? Обновить", command=
49           apply_filters_and_redraw, width=12, bg="#e0e0e0").pack(side=tk.
50           RIGHT, padx=4)
51 tk.Button(ctrl_frame, text="?? Экспорт", command=export_plot, width
52           =12, bg="#e0e0e0").pack(side=tk.RIGHT, padx=4)
53
54 if __name__ == '__main__':
55     df_work = df_processed.copy()
56     plot_line()
57     root.mainloop()

```

4 Скриншоты работы программы

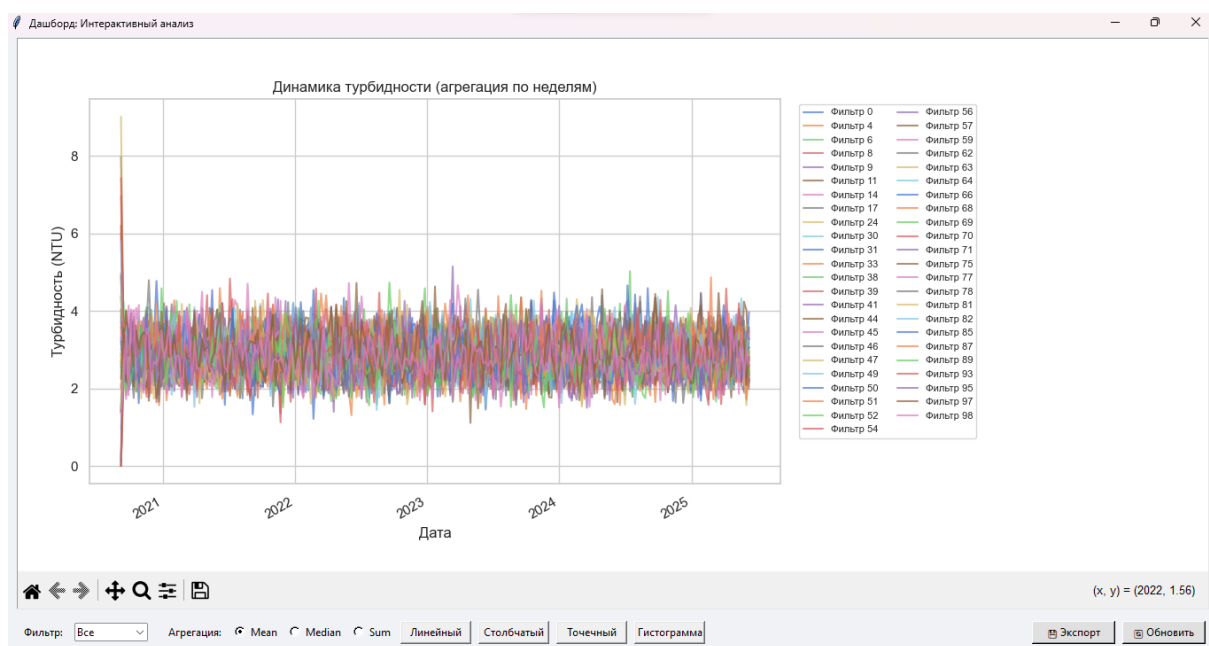


Рис. 1: Линейный график: динамика турбидности по всем фильтрам

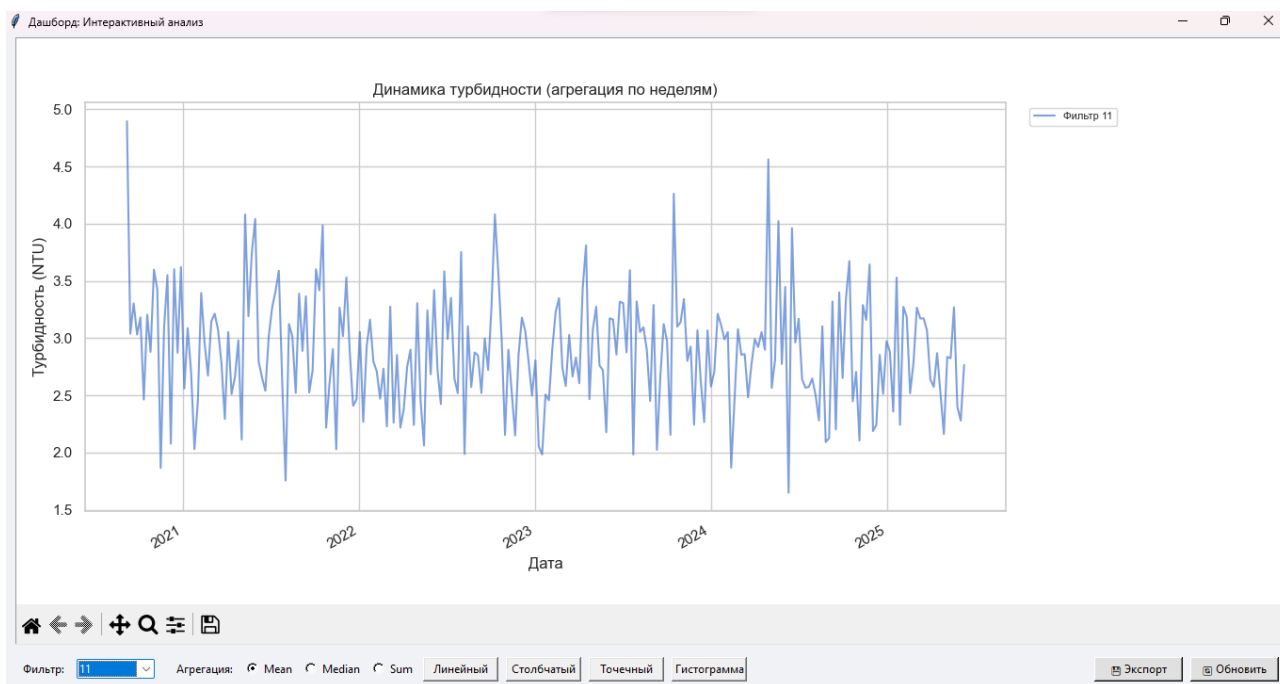


Рис. 2: Линейный график: динамика турбидности по одному фильтру

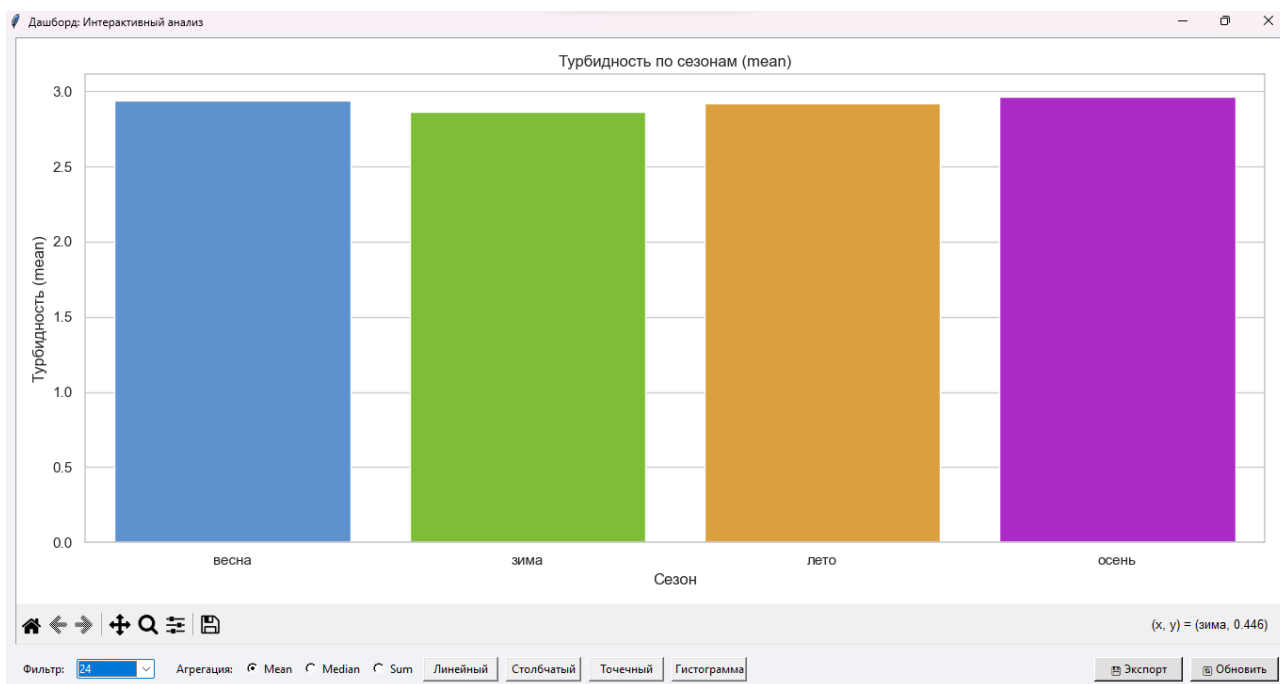


Рис. 3: Столбчатая диаграмма: агрегация турбидности по сезонам

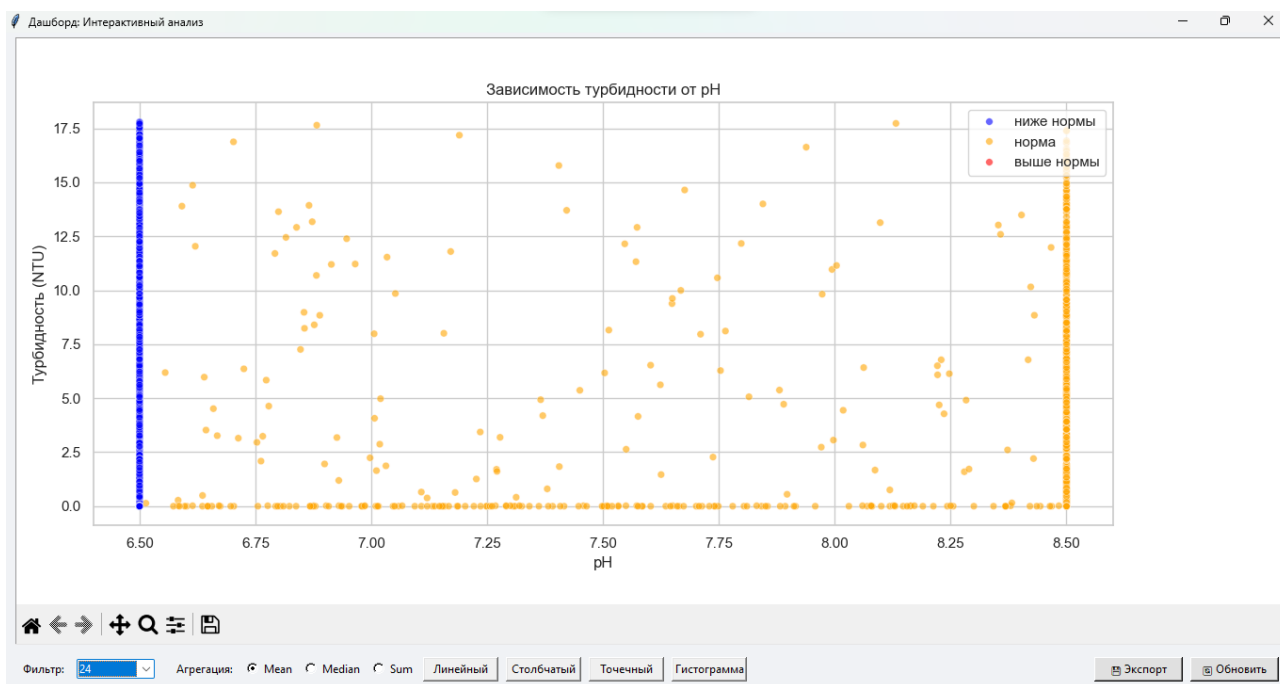


Рис. 4: Точечная диаграмма: зависимость turb от pH с цветовым кодированием статуса

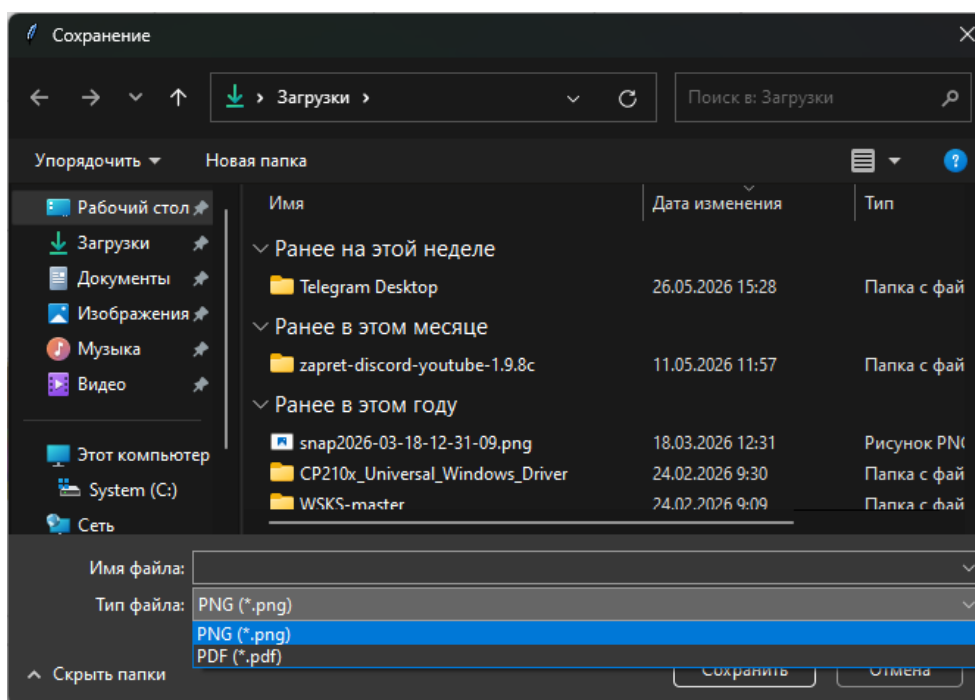


Рис. 5: Диалог сохранения в PNG/PDF

5 Анализ результатов

В ходе выполнения лабораторной работы были получены следующие аналитические инсайты и выявлены системные ограничения:

Полученные инсайты:

- Применение скользящего среднего (`window=35`) эффективно сглаживает шумовые выбросы турбидности, позволяя чётко выделить долгосрочные тренды эффективности фильтрации. Резкие пики на сырых данных соответствуют кратковременным перегрузкам насосов, а не системному ухудшению качества воды.
- Независимая обработка масок аномалий (IQR для `turb`, `clip` для `ph`) предотвращает каскадное искажение данных. Цветовое кодирование `ph_status` на scatter-plot наглядно демонстрирует отсутствие прямой линейной корреляции между кислотностью и мутностью, что подтверждает необходимость раздельного контроля параметров.
- Агрегация по сезонам выявляет, что максимальные значения турбидности приходятся на весенний период (паводковые нагрузки), что коррелирует с увеличением `flow`.

Выявленные ограничения:

- Архитектура Tkinter является однопоточной. При обработке датасетов свыше 10^7 строк пересчёт `groupby()` и отрисовка Seaborn могут вызывать кратковременные «фризы» интерфейса.
- Статический рендеринг Matplotlib/Seaborn не поддерживает аппаратное ускорение GPU, что ограничивает частоту обновления в режимах реального времени (Real-time streaming).
- Использование `canvas.draw_idle()` ставит задачу в очередь событий, но не гарантирует мгновенного отражения данных при частых изменениях ползунков или фильтров.

Пути масштабирования решения:

1. Перенос тяжёлых вычислений в фоновый поток с использованием `root.after()` или `queue.Queue` для безопасной передачи результатов в GUI-поток.
2. Замена Seaborn на Plotly или Bokeh для нативной поддержки WebGL-ускорения и интерактивного масштабирования без потери производительности.
3. Интеграция Dask DataFrame или Polars для распределённой обработки данных, выходящих за пределы RAM, с сохранением логики фильтрации на уровне API.

6 Выводы

В ходе выполнения лабораторной работы были успешно освоены ключевые технологии создания интерактивных аналитических панелей в экосистеме Python:

- Реализована событийно-ориентированная архитектура с использованием callback-функций и цикла событий `mainloop()`, что обеспечило мгновенный отклик интерфейса на действия пользователя.
- Освоено встраивание графических объектов Matplotlib в оконную систему Tkinter через адаптер `FigureCanvasTkAgg`, включая корректную работу панели навигации (`NavigationToolbar2Tk`).
- На практике применены методы динамической фильтрации (`df.loc[]`), агрегации (`groupby().agg()`), оконных функций (`rolling()`) и биннинга (`pd.cut()`) библиотеки pandas. Соблюдено правило обязательного копирования данных (`.copy()`) для предотвращения `SettingWithCopyWarning` и искажения исходного датасета.
- Разработано устойчивое процедурное приложение без использования ООП-паттернов, что упростило отладку и чтение кода.

Трудности: Основным вызовом стала синхронизация состояния данных между этапом предобработки и этапом отрисовки. Некорректное использование ссылок на DataFrame приводило к накоплению артефактов на графиках. Проблема была решена строгим разделением `df_processed` (статический эталон) и `df_work` (рабочая копия), а также использованием `fig.clear()` перед каждой отрисовкой.

Пути улучшения: Добавление логгирования событий, внедрение прогресс-баров для длительных вычислений, поддержка темной темы интерфейса и автоматическое сохранение состояния фильтров между сессиями.

7 Ссылка на репозиторий

Исходный код проекта, скрипты генерации данных и инструкции по запуску доступны по ссылке:

<https://github.com/GIKExe/Subject/tree/main/Proga/laba7>